

iOS面试题合集（算法篇）

iOSer iOSer 5月19日

收录于话题

#BAT大厂面试题合集

9个

欢迎点击上方蓝字“iOSer”关注我们
再点击右上角“...”菜单，选择“设为星标”
我们一起精进、成长！

本栏目将持续更新--请iOS的小伙伴关注我们!

做这个的初心是希望能巩固自己的基础知识，

当然也希望能帮助更多的开发者，

可以加入我们的技术交流群：**834688868**，群里每天都会有干货、技术动向、职业生涯、行业热点、职场趣事等一切有关于程序员的内容分享。

不管你是大牛还是小白都欢迎入驻，分享BAT，阿里面试题、面试经验，讨论技术，大家一起交流学习成长！

目录：

- 1.时间复杂度 / 空间复杂度
- 2.常用的排序算法有哪些？
- 3.字符串反转
- 4.链表反转（头差法）
- 5.如何查找第一个只出现一次的字符（Hash查找）
- 6.如何查找两个子视图的共同父视图？
- 7.无序数组中的中位数(快排思想)
- 8.如何给定一个整数数组和一个目标值，找出数组中和为目标值的两个数。

1.时间复杂度 / 空间复杂度

1, 时间复杂度

时间频度

一个算法执行所耗费的时间,从理论上是不能算出来的,必须上机运行测试才能知道.但我们不可能也没有必要对每个算法都上机测试,

只需知道哪个算法花费的时间多,哪个算法花费的时间少就可以了.并且一个算法花费的时间与算法中语句的执行次数成正比例,

哪个算法中语句执行次数多,它花费时间就多.一个算法中的语句执行次数称为语句频度或时间频度.记为 $T(n)$.

时间复杂度

一般情况下,算法中基本操作重复执行的次数是问题规模 n 的某个函数,用 $T(n)$ 表示,若有某个辅助函数 $f(n)$,使得当 n 趋近于无穷大时, $T(n)/f(n)$ 的极限值为不等于零的常数,则称 $f(n)$ 是 $T(n)$ 的同数量级函数.记作 $T(n)=O(f(n))$,称 $O(f(n))$ 为算法的渐进时间复杂度,简称时间复杂度.

在各种不同算法中,若算法中语句执行次数为一个常数,则时间复杂度为 $O(1)$,另外,在时间频度不相同,时间复杂度有可能相同,如 $T(n)=n^2+3n+4$ 与 $T(n)=4n^2+2n+1$ 它们的频度不同,但时间复杂度相同,都为 $O(n^2)$.

按数量级递增排列,常见的时间复杂度有:

$O(1)$ 称为常量级, 算法的时间复杂度是一个常数。

$O(n)$ 称为线性级, 时间复杂度是数据量 n 的线性函数。

$O(n^2)$ 称为平方级, 与数据量 n 的二次多项式函数属于同一数量级。

$O(n^3)$ 称为立方级, 是 n 的三次多项式函数。

$O(\log n)$ 称为对数级, 是 n 的对数函数。

$O(n \log n)$ 称为介于线性级和平方级之间的一种数量级

$O(2^n)$ 称为指数级，与数据量 n 的指数函数是一个数量级。

$O(n!)$ 称为阶乘级，与数据量 n 的阶乘是一个数量级。

它们之间的关系是： $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$ ，随着问题规模 n 的不断增大，上述时间复杂度不断增大，算法的执行效率越低。

2.空间复杂度

评估执行程序所需的存储空间。可以估算出程序对计算机内存的使用程度。不包括算法程序代码和所处理的数据本身所占空间部分。通常用所使用额外空间的字节数表示。其算法比较简单，记为 $S(n) = O(f(n))$ ，其中， n 表示问题规模。

2.常用的排序算法有哪些？

选择排序、冒泡排序、插入排序三种排序算法可以总结为如下：

都将数组分为已排序部分和未排序部分。

选择排序将已排序部分定义在左端，然后选择未排序部分的最小元素和未排序部分的第一个元素交换。

冒泡排序将已排序部分定义在右端，在遍历未排序部分的过程执行交换，将最大元素交换到最右端。

插入排序将已排序部分定义在左端，将未排序部分的第一个元素插入到已排序部分合适的位置。



```
/**
 * 【选择排序】：最值出现在起始端
 *
 * 第1趟：在n个数中找到最小(大)数与第一个数交换位置
 * 第2趟：在剩下n-1个数中找到最小(大)数与第二个数交换位置
 * 重复这样的操作...依次与第三个、第四个...数交换位置
```

```

    * 第n-1趟，最终可实现数据的升序（降序）排列。
    *
    */
void selectSort(int *arr, int length) {
    for (int i = 0; i < length - 1; i++) { //趟数
        for (int j = i + 1; j < length; j++) { //比较次数
            if (arr[i] > arr[j]) {
                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }
}

/**
 * 【冒泡排序】：相邻元素两两比较，比较完一趟，最大值出现在末尾
 * 第1趟：依次比较相邻的两个数，不断交换（小数放前，大数放后）逐个推进，最大值出现在末尾
 * 第2趟：依次比较相邻的两个数，不断交换（小数放前，大数放后）逐个推进，最大值出现在末尾
 * .....
 * 第n-1趟：依次比较相邻的两个数，不断交换（小数放前，大数放后）逐个推进，最大值出现在末尾
 */
void bubbleSort(int *arr, int length) {
    for (int i = 0; i < length - 1; i++) { //趟数
        for (int j = 0; j < length - i - 1; j++) { //比较次数
            if (arr[j] > arr[j+1]) {
                int temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
}

/**
 * 折半查找：优化查找时间（不用遍历全部数据）
 *
 * 折半查找的原理：
 * 1> 数组必须是有序的
 * 2> 必须已知min和max（知道范围）
 * 3> 动态计算mid的值，取出mid对应的值进行比较
 */

```

```

*    4> 如果mid对应的值大于要查找的值，那么max要变小为mid-1
*    5> 如果mid对应的值小于要查找的值，那么min要变大为mid+1
*
*/

// 已知一个有序数组， 和一个key， 要求从数组中找到key对应的索引位置
int findKey(int *arr, int length, int key) {
    int min = 0, max = length - 1, mid;
    while (min <= max) {
        mid = (min + max) / 2; //计算中间值
        if (key > arr[mid]) {
            min = mid + 1;
        } else if (key < arr[mid]) {
            max = mid - 1;
        } else {
            return mid;
        }
    }
    return -1;
}

```

3.字符串反转



```

void char_reverse (char *cha) {

    // 定义头部指针
    char *begin = cha;
    // 定义尾部指针
    char *end = cha + strlen(cha) - 1;

    while (begin < end) {

        char temp = *begin;
        *(begin++) = *end;

```

```
        *(end--) = temp;
    }
}
```

4.链表反转（头差法）

链表反转（头差法）

- .h声明文件



```
#import <Foundation/Foundation.h>

// 定义一个链表
struct Node {
    int data;
    struct Node *next;
};

@interface ReverseList : NSObject

// 链表反转
struct Node* reverseList(struct Node *head);

// 构造一个链表
struct Node* constructList(void);

// 打印链表中的数据
void printList(struct Node *head);

@end
```

- .m实现文件



```
#import "ReverseList.h"

@implementation ReverseList

struct Node* reverseList(struct Node *head)
{
    // 定义遍历指针，初始化为头结点
    struct Node *p = head;

    // 反转后的链表头部
    struct Node *newH = NULL;

    // 遍历链表
    while (p != NULL) {

        // 记录下一个结点
        struct Node *temp = p->next;
        // 当前结点的next指向新链表头部
        p->next = newH;
        // 更改新链表头部为当前结点
        newH = p;
        // 移动p指针
        p = temp;
    }

    // 返回反转后的链表头结点
    return newH;
}

struct Node* constructList(void)
{
    // 头结点定义
    struct Node *head = NULL;
    // 记录当前尾结点
    struct Node *cur = NULL;

    for (int i = 1; i < 5; i++) {
        struct Node *node = malloc(sizeof(struct Node));
        node->data = i;
    }
}
```

```
// 头结点为空，新结点即为头结点
if (head == NULL) {
    head = node;
}
// 当前结点的next为新结点
else{
    cur->next = node;
}

// 设置当前结点为新结点
cur = node;
}

return head;
}

void printList(struct Node *head)
{
    struct Node* temp = head;
    while (temp != NULL) {
        printf("node is %d \n", temp->data);
        temp = temp->next;
    }
}

@end
```

5.如何查找第一个只出现一次的字符（Hash查找）

- .h声明文件



```
#import <Foundation/Foundation.h>
```



```
@interface HashFind : NSObject

// 查找第一个只出现一次的字符
char findFirstChar(char* cha);

@end

1
2
3
4
5
6
7
8
9
```

- .m实现文件



```
#import "HashFind.h"

@implementation HashFind

char findFirstChar(char* cha)
{
    char result = '\0';

    // 定义一个数组 用来存储各个字母出现次数
    int array[256];

    // 对数组进行初始化操作
    for (int i=0; i<256; i++) {
        array[i] = 0;
    }

    // 定义一个指针 指向当前字符串头部
    char* p = cha;
    // 遍历每个字符
```

```

while (*p != '\0') {
    // 在字母对应存储位置 进行出现次数+1操作
    array[*p]++;
}

// 将P指针重新指向字符串头部
p = cha;
// 遍历每个字母的出现次数
while (*p != '\0') {
    // 遇到第一个出现次数为1的字符，打印结果
    if (array[*p] == 1)
    {
        result = *p;
        break;
    }
    // 反之继续向后遍历
    p++;
}

return result;
}

@end

```

6.如何查找两个子视图的共同父视图？

- h声明文件



```

#import <Foundation/Foundation.h>
#import <UIKit/UIKit.h>
@interface CommonSuperFind : NSObject

// 查找两个视图的共同父视图
- (NSArray<UIView *> *)findCommonSuperView:(UIView *)view other:(UIView *)viewOther;

```

@end

- m实现文件



```
#import "CommonSuperFind.h"
```

```
@implementation CommonSuperFind
```

```
- (NSArray <UIView *> *)findCommonSuperView:(UIView *)viewOne  
other:(UIView *)viewOther
```

```
{
```

```
    NSMutableArray *result = [NSMutableArray array];
```

```
    // 查找第一个视图的所有父视图
```

```
    NSArray *arrayOne = [self findSuperViews:viewOne];
```

```
    // 查找第二个视图的所有父视图
```

```
    NSArray *arrayOther = [self findSuperViews:viewOther];
```

```
    int i = 0;
```

```
    // 越界限制条件
```

```
    while (i < MIN((int)arrayOne.count, (int)arrayOther.count)  
) {
```

```
        // 倒序方式获取各个视图的父视图
```

```
        UIView *superOne = [arrayOne objectAtIndex:index:arrayOne.co  
unt - i - 1];
```

```
        UIView *superOther = [arrayOther objectAtIndex:index:arrayOt  
her.count - i - 1];
```

```
        // 比较如果相等 则为共同父视图
```

```
        if (superOne == superOther) {  
            [result addObject:superOne];  
            i++;  
        }
```

```
        // 如果不相等，则结束遍历
```

```
        else{  
            break;  
        }  
    }
```

```
    }

    return result;
}

- (NSArray <UIView *> *)findSuperViews:(UIView *)view
{
    // 初始化为第一父视图
    UIView *temp = view.superview;
    // 保存结果的数组
    NSMutableArray *result = [NSMutableArray array];
    while (temp) {
        [result addObject:temp];
        // 顺着superview指针一直向上查找
        temp = temp.superview;
    }
    return result;
}

@end
```

7.无序数组中的中位数(快排思想)

- h声明文件



```
#import <Foundation/Foundation.h>

@interface MedianFind : NSObject

// 无序数组中位数查找
int findMedian(int a[], int aLen);

@end
```

- m实现文件



```
#import "MedianFind.h"

@implementation MedianFind

//求一个无序数组的中位数
int findMedian(int a[], int aLen)
{
    int low = 0;
    int high = aLen - 1;

    int mid = (aLen - 1) / 2;
    int div = PartSort(a, low, high);

    while (div != mid)
    {
        if (mid < div)
        {
            //左半区间找
            div = PartSort(a, low, div - 1);
        }
        else
        {
            //右半区间找
            div = PartSort(a, div + 1, high);
        }
    }
    //找到了
    return a[mid];
}

int PartSort(int a[], int start, int end)
{
    int low = start;
    int high = end;

    //选取关键字
    int key = a[end];
```

```
while (low < high)
{
    //左边找比key大的值
    while (low < high && a[low] <= key)
    {
        ++low;
    }

    //右边找比key小的值
    while (low < high && a[high] >= key)
    {
        --high;
    }

    if (low < high)
    {
        //找到之后交换左右的值
        int temp = a[low];
        a[low] = a[high];
        a[high] = temp;
    }
}

int temp = a[high];
a[high] = a[end];
a[end] = temp;

return low;
}

@end
```

8.如何给定一个整数数组和一个目标值，找出数组中和为目标值的两个数。



```

- (void)viewDidLoad {

    [super viewDidLoad];

    NSArray *oriArray = @[2], [3], [6], [7], [22], [12]];

    BOOL isHaveNums = [self twoNumSumWithTarget:9 Array:oriArray];

    NSLog(@"%d", isHaveNums);
}

- (BOOL)twoNumSumWithTarget:(int)target Array:(NSArray<NSNumber *> *)array {

    NSMutableArray *finalArray = [NSMutableArray array];

    for (int i = 0; i < array.count; i++) {

        for (int j = i + 1; j < array.count; j++) {

            if ([array[i] intValue] + [array[j] intValue] == target) {

                [finalArray addObject:array[i]];
                [finalArray addObject:array[j]];
                NSLog(@"%@", finalArray);

                return YES;
            }
        }
    }

    return NO;
}

```

部分总结的算法面试题就到这里了，

以上内容对你有帮助的话，记得关注我们！

加入我们的iOS高级技术交流QQ群：**834688868**，群里每天都会有干货、技术动向、职业生涯、行业热点、职场趣事等一切有关于程序员的内容分享。

不管你是大牛还是小白都欢迎入驻，分享BAT，阿里面试题、面试经验，讨论技术，大家一起交流学习成长！

文章来源于网络，已尽可能标明作者以及来源，文章内容为作者独立观点，不代表本公众号立场，因文章侵权本公众号不承担任何法律及连带责任，如有侵权，请联系我们删除。



觉得不错的话，别忘了点个"在看"哦~ 📌

收录于话题 #BAT大厂面试题合集 · 9个

上一篇

iOS面试题合集（内存管理篇）

下一篇

iOS面试题合集（数据结构篇）

喜欢此内容的人还喜欢

iOS技术人员如何做晋升答辩？

iOSer

驻阿美军走得如此狼狈？断电断水垃圾遍地，阿富汗军人转身就投降

冰汝看美国

镇魂井？别再神话林生斌

灵魂有香气的女子